

ForensiX-AI

Complete Project Execution Roadmap

AI-Powered Digital Memory Search Engine & Windows Cybersecurity Investigation Platform

Purpose: Shared technical execution guide for the team and AI coding assistants. It defines the current state, target architecture, implementation order, testing plan, scope boundaries, and immediate next steps.

Current status	Project foundation complete; database implementation is next
Target platform	Single Windows 10/11 endpoint
Backend	Python + Flask/FastAPI
Database	SQLite initially; PostgreSQL may be used for scalable deployment
AI	Ollama local LLM + machine-learning baseline/correlation
Frontend	React.js
Integrity	SHA-256 hash chaining / tamper-evident logging

Implementation rule: Build and validate the data pipeline first. Do not start with the LLM or dashboard; AI depends on clean, structured event data.

1. Current Project Structure

```
ForensiX-AI/  
├── backend/  
│   ├── ai/  
│   ├── collectors/  
│   ├── database/  
│   ├── __init__.py  
│   ├── db.py  
│   ├── models.py  
│   ├── schema.py  
│   └── forensix_ai.db  
├── integrity/  
├── processing/  
├── __init__.py  
├── app.py  
├── data/  
├── frontend/  
├── tests/  
├── .gitignore  
├── README.md  
└── requirements.txt
```

The `__pycache__` directories are generated automatically by Python and should be excluded from GitHub through `.gitignore`.

2. Target Architecture

Data Collection Layer

Windows Event Logs, Browser History, File Activity, USB Device Events and Windows Defender Alerts from one Windows endpoint.

Processing Core

Event normalization, centralized storage, AI/rule-based correlation, behavioral baseline and SHA-256 hash-chain verification.

Output / Investigation Layer

Natural-language investigation, incident analysis and structured report generation.

Final / User Interface Layer

Investigator dashboard showing timelines, alerts, investigation results, integrity status and reports.

Windows Endpoint → Collect → Normalize → Store → Correlate → Verify → Investigate → Report → Dashboard

3. Common Event Schema

Field	Purpose
event_id	Unique event identifier.
timestamp	Time the activity/event occurred.
source	Windows, Browser, File, USB or Defender.
event_type	Normalized type such as LOGIN, FILE_CREATE, USB_INSERT, WEBSITE_VISIT.
user_name	Relevant Windows user/account, when available.
process_name	Relevant application/process, when available.
path_or_url	File path, URL or source-specific identifier, when applicable.
description	Human-readable event description.
severity	Low / Medium / High / Critical according to defined rules.

Field	Purpose
event_hash	SHA-256 hash for integrity chaining.
previous_hash	Previous event hash in the chain.

PHASE 0 — Project Foundation [DONE]

- GitHub repository and project structure.
- Python package `__init__.py` files.
- `database/`, `collectors/`, `processing/`, `integrity/`, `ai/`, `frontend/`, `tests/` and `data/` folders.
- `requirements.txt`, `README.md` and `.gitignore`.
- SQLite file: `forensix_ai.db`.

Foundation only; no major functionality yet.

PHASE 1 — Database & Event Schema [NEXT]

- Implement `database/db.py` and SQLite connection.
- Define `models.py` for the common event structure.
- Implement `schema.py` and create the events table.
- Test event insertion, retrieval and querying.

Deliverable: working centralized event database.

PHASE 2 — Event Normalization [PENDING]

- Create `processing/normalizer.py`.
- Convert every raw collector event into the common schema.
- Validate timestamps and required fields.
- Handle missing/optional source fields safely.

Deliverable: Raw Event → Normalized Event.

PHASE 3 — Windows Event Log Collector [PENDING]

- Create `collectors/windows_events.py`.
- Use `pywin32` for supported Windows Event Logs.
- Start with relevant system/security/application events.
- Normalize and store events.
- Test using controlled Windows activity.

Deliverable: first working telemetry collector.

PHASE 4 — Windows Defender Collector [PENDING]

- Create `collectors/defender.py`.
- Collect supported Defender threat/security events.
- Normalize and store them.
- Test with safe controlled scenarios.

Deliverable: Defender events in the central database.

PHASE 5 — File Activity Collector [PENDING]

- Create `collectors/file_activity.py`.
- Use `watchdog`.
- Detect create, modify, delete and rename activity.
- Record timestamps, paths and relevant metadata.
- Test on a controlled directory.

Deliverable: supported real-time file activity monitoring.

PHASE 6 — USB Device Collector [PENDING]

- Create collectors/usb.py.
- Use WMI/supported Windows telemetry.
- Detect device insertion/removal.
- Store available device information and timestamps.
- Test controlled USB connect/disconnect.

Deliverable: USB event collection.

PHASE 7 — Browser History Collector [PENDING]

- Create collectors/browser.py.
- Initially support Chrome/Edge/Firefox where feasible.
- Read available URLs and timestamps.
- Use safe copy/read handling for locked browser databases.
- Normalize and store events.

Deliverable: browser history events in the central database.

PHASE 8 — Integrated Processing Pipeline [PENDING]

- Connect all collectors to normalization and storage.
- Handle duplicates, malformed events and missing values.
- Check timestamps/source labels.
- Measure storage and collection overhead.

Deliverable: integrated telemetry pipeline.

PHASE 9 — Hash-Chain Integrity [PENDING]

- Create integrity/hasher.py and verifier.py.
- Generate SHA-256 hashes.
- Include previous hash in the next hash calculation.
- Verify the chain.
- Test controlled modification/deletion detection.

Deliverable: tamper-evident logging and verification.

PHASE 10 — Event Correlation [PENDING]

- Start with deterministic/rule-based correlation.
- Define event relationships and time windows.
- Group related events into potential incidents.
- Store incident-to-event relationships.
- Evaluate controlled scenarios.

Deliverable: related events → potential incident.

PHASE 11 — Behavioral Baseline [PENDING]

- Create ai/behavior.py.
- Collect baseline features such as normal active periods and recurring activity.
- Start with statistical/ML methods.
- Calculate deviation scores.

- Handle cold-start and alert thresholds.

Deliverable: meaningful activity deviations.

PHASE 12 — Natural-Language Investigation [PENDING]

- Integrate Ollama after database/search is stable.
- Create ai/llm.py and query_engine.py.
- Translate natural-language requests into safe searches/filters.
- Return only evidence retrieved from the database.
- Prevent invented evidence.

Deliverable: plain-English investigation over stored events.

PHASE 13 — Incident Reports [PENDING]

- Create ai/report_generator.py.
- Generate incident summaries, timelines and relevant evidence.
- Include integrity verification status.
- Export practical HTML/PDF reports.

Deliverable: structured investigation report.

PHASE 14 — Backend API [PENDING]

- Connect modules using Flask or FastAPI.
- Provide events, incidents, timeline, alerts, search, investigation, reports and integrity endpoints.
- Add validation and error handling.
- Test endpoints independently.

Deliverable: stable frontend/backend interface.

PHASE 15 — Investigator Dashboard [PENDING]

- Build React dashboard.
- Overview/dashboard.
- Event timeline.
- Alerts and suspicious activity.
- Natural-language investigation.
- Incident details and reports.
- Integrity status.

Deliverable: complete investigator interface.

PHASE 16 — Testing & Evaluation [PENDING]

- Unit test collectors and database.
- Test normalization.
- Test hash-chain tamper detection.
- Test correlation and behavioral scenarios.
- Evaluate LLM grounding.
- Test API/UI integration.
- Measure accuracy, latency and resource use.

Deliverable: evidence that the prototype works.

PHASE 17 — Documentation & Final Delivery [PENDING]

- Finalize report.
- Prepare architecture, database, DFD/use-case/sequence diagrams.
- Document test cases and results.
- Document limitations/future scope.
- Prepare screenshots, demo, PPT and viva material.
- Update README.

Deliverable: final project package.

4. Database Direction

Table	Purpose	Phase
events	Central normalized security events	1
incidents	Correlated groups of related events	10
behavior_profile	Behavioral baseline data	11
reports	Generated report metadata/content	13

5. Implementation Rules

- Follow the roadmap order unless a technical dependency requires a change.
- Do not build the LLM first; the database and event pipeline come first.
- Each collector must be independently testable.
- Every collector must output the common event schema.
- Keep Windows-specific collection code inside collectors/.
- Keep database access inside database/.
- Keep hashing/verification inside integrity/.
- Keep normalization inside processing/.
- Keep AI/ML/LLM code inside ai/.
- Do not claim to collect literally everything on the computer; collect available and authorized telemetry from supported sources.
- Do not claim deleted-file recovery, disk imaging, RAM forensics or network-wide monitoring unless implemented.
- Describe SHA-256 chaining as tamper-evident hash-chain logging, not a full blockchain.
- LLM responses must be grounded in retrieved evidence and must not invent events.
- Use controlled, privacy-safe test scenarios.
- Make small, meaningful Git commits and test each phase before moving on.

6. Controlled Test Scenarios

Scenario	Expected result
Windows login	Normalized login/security event is captured.
Create/modify/delete test file	Supported file activity is captured with path/time.
USB connect/disconnect	USB insertion/removal is recorded.
Visit a test URL	Available browser URL/timestamp is recorded.
Controlled Defender event	Supported Defender event is collected.
Related multi-source activity	Correlation groups relevant events into a potential incident.
Modify stored event	Hash verification detects integrity failure.
Natural-language query	Relevant stored evidence is returned without invented events.
Unusual activity	Baseline generates a deviation signal when threshold is met.

Scenario	Expected result
Generate report	Report includes relevant evidence and integrity status.

7. Scope Boundaries

- Current target: working Windows prototype for one endpoint.
- Out of scope: deleted-file recovery/data carving, full disk imaging, RAM acquisition, network-wide monitoring.
- The system cannot recreate telemetry that was never collected.
- Do not claim universal legal admissibility.
- Do not claim complete visibility into every possible Windows/browser artifact.

8. Estimated Development Schedule

Stage	Approx. time
Foundation + database	2–3 days
Normalization	1–2 days
Windows + Defender	3–5 days
File + USB + Browser	6–8 days
Processing + integrity	4–5 days
Correlation + baseline	6–9 days
NLP/LLM + reports	5–8 days
API + dashboard	6–10 days
Testing + documentation	5–7 days
Overall prototype target	~4–6 weeks of consistent work

9. Immediate Next Action

Next: Implement backend/database/db.py → connect Python to forensix_ai.db → test the connection → implement schema.py → create the events table → define models.py/common event structure.

This roadmap should be treated as the shared execution reference when assigning work to teammates or AI coding assistants.